DO NOW — RECALL

Merge sort: trace and implement

Name: _____ Class: _____ Date: _____

1. In insertion sort, what happens to values greater than the key in the sorted prefix?

2. After one ascending bubble-sort pass on [4,1,3], what remains?

3. What is the length of int[] a={8,5,2,1}?

4. Insertion sort has sorted prefix [2,4,6,8] and next key 5. Trace the prefix after inserting the key, listing the values shifted right in order.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

They shift one position right to make space.

2. Model answer

[1,3,4].

3. Model answer

4.

4. Model answer

Result: [2,4,5,6,8]. Shift 8, then 6 right; insert 5 after 4.

Vocabulary — know these terms

run — A consecutive sorted block.

merge — Combine sorted sequences while preserving order.

pointer — Position of the next candidate in a run.

bottom-up — Build sorted runs iteratively from small to large.

O(n log n) — Common time bound for comparison-based merge sort.

NEW LEARNING

Read, annotate and explain

Start with ordered runs

At each merge stage, both input runs must already be sorted.

Compare front values

Append the smaller current value, then advance that run's pointer.

Copy the remainder

When a run finishes, copy the remainder. This code is the merge helper: top-down merge sort first recursively splits to length 0/1; bottom-up merge sort instead grows the run width.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Read the code; predict its effect



Worked algorithm states: LEFT RUN 2,7; RIGHT RUN 3,6; MERGED 2,3,6,7

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Check the complete trace

Singletons return; merge [1,6] and [2,4]; final [1,2,4,6]. copyRange copies the stated slice, excluding stop; MergeSorted compares fronts and copies the remaining tail. These are defined helpers, not built-in OCR commands.

IMPROVE: Explain one correction to your own reasoning.

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

Two unsorted arrays can be passed to the standard merge step and always yield sorted output.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

I DO — WORKED EXAMPLE

Follow the worked method

Trace recursive merge sort on [8,3,5,2]: split, reach the base case, then merge on return.

Split until each call is trivial

Split [8,3,5,2] into [8,3] and [5,2], then into [8],[3],[5],[2]. Each length-one call returns unchanged; an empty input also returns immediately.

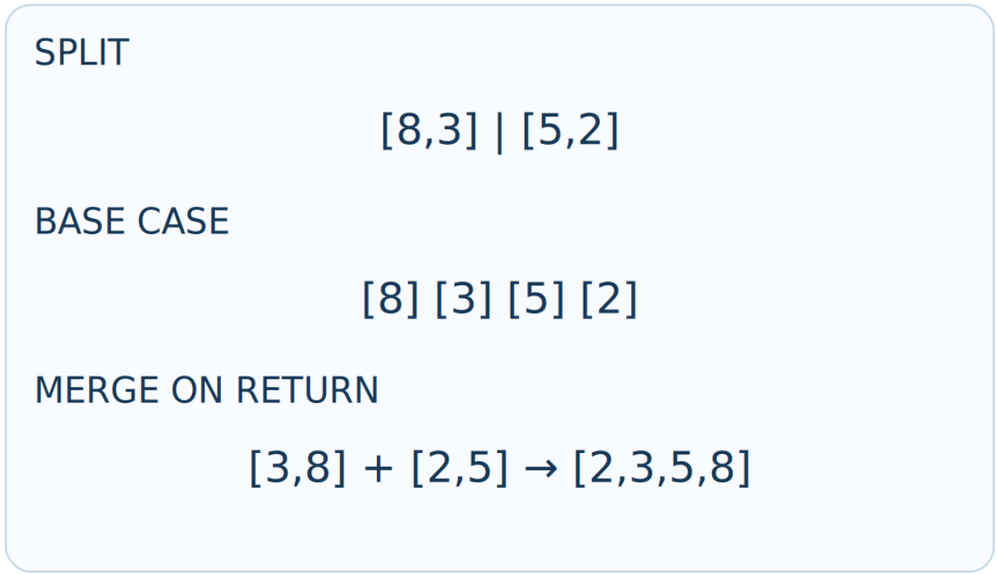
Return through the merges

Merge [8] with [3]→[3,8]; [5] with [2]→[2,5]. Merge fronts 2,3,5, then copy8: [2,3,5,8]. Each helper input is already sorted. The outline uses end-exclusive copyRange and the merge loop above.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace recursive merge sort on [8,3,5,2]: split, reach the base case, then merge on return.



Recursive merge sort: split 8 3 5 2 to singletons; merge 3 8 and 2 5; result 2 3 5 8

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Explain the O(n log n) time complexity of merge sort.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Merge [2,6] with [1,5].

Question 2

Recursively merge-sort [4,1,3,2]. Show both split levels, the base case and every merge on return.

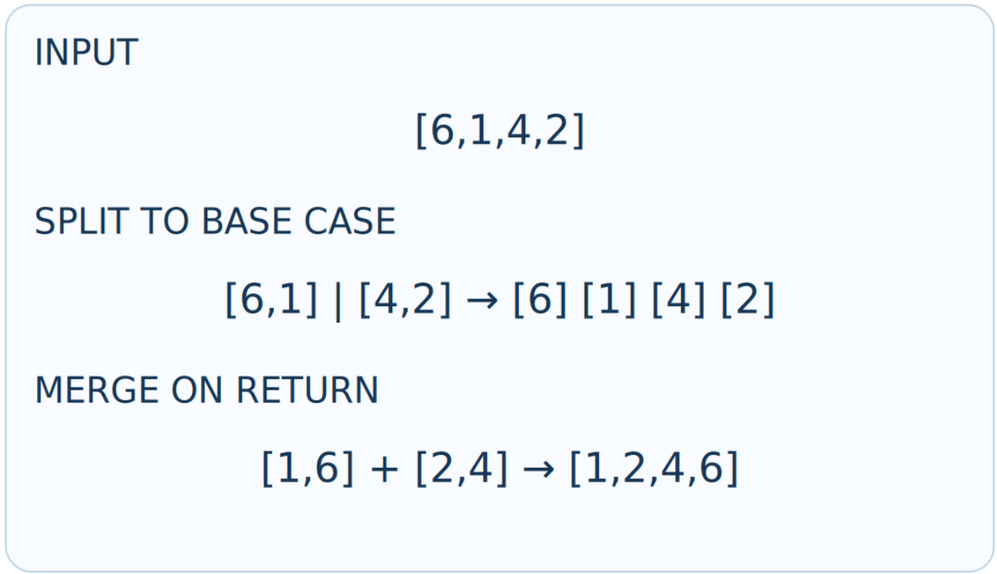
WE DO — GUIDED PRACTICE

Try → Discuss → Improve

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Use this recursive outline to sort [6,1,4,2]. Predict the split calls, intermediate merges and result.



Recursive merge sort on 6 1 4 2: split to singletons and merge on return

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Predict the call and return order Pseudocode outline: MergeSort(a): if a.length <= 1: return a mid = a.length DIV 2 L = MergeSort(copyRange(a,0,mid)) R =…

DISCUSS — Compare the sequence of steps or states. Identify the first difference in your methods and justify which step is correct.

IMPROVE — Write one precise correction or stronger explanation in your own answer.